Guión de trabajo tutorizado

Bases de datos In-Memory II

Gestión y visualización de datos masivos

Profesor Dr. Domingo Savio Rodríguez Baena
Máster Big Data & Business Analytics (MBDA)





Alumno: ÁLVARO RAMOS BERNABÉ

ENUNCIADO DE LA ACTIVIDAD

Guion oficial: «MBDA-BasesDeDatosInMemoryII-Actividad.pdf» (Campus Virtual UPO).

Objetivos

- Probar de forma sencilla la ventaja de tener toda la información en memoria
- Comparar rendimientos con el sistema tradicional

Ejercicios

E1. (180 minutos – 5 puntos) Experimento con MONGODB con motor de base de datos estándar.

Vamos a realizar un estudio temporal con operaciones de lectura y escritura. El objetivo es medir tiempos para distintos tipos de operaciones y distintos tamaños de colección.

Estas son las distintas operaciones con las que vamos a experimentar:

I1: inserción de documentos en la colección, gracias a la función:

```
function populatePhones(area,start,stop) {
for(var i=start; i < stop; i++) {
var country = 1 + ((Math.random() * 8) << 0); var num = (country * 1e10) + (area * 1e7) + i;

db.phones.insertOne({
_id: num,
components: {
country: country,
area: area,
prefix: (i * 1e-4) << 0,
number: i
},
display: "+" + country + " " + area + "-" + i
});
}
}
```

Como recordaréis, dicha función ya fue utilizada en la actividad de MongoDB. La utilizaremos de nuevo para hacer distintas cargas masivas de documentos en nuestra base de datos.

El experimento se va a desarrollar a partir de los siguientes pasos:

1º) Realizar la inserción de un número concreto de documentos y medir sus tiempos.

La cantidad en cuestión dependerá de los recursos hardware de los que disponga el alumno. Es decir, si se cuenta con un equipo moderno y rápido, vamos a necesitar muchos documentos para poder registrar tiempos distintos de 0 milisegundos. Con un equipo más antiguo, quizás sean necesarios menos documentos.

Indicar el comando utilizado y el número de documentos insertado, al igual que el tiempo medido.

2º) Ejecución de las consultas.

Tenemos un conjunto de tipo de consultas:

Q1: Consulta de documento específico a partir de atributo sin indexar: Buscaremos un documento en concreto, basándonos en un atributo que no esté indexado. De esta forma, forzaremos a MONGODB a buscar en toda la colección.

Q2: Consulta de documento específico a partir de atributo indexado: Ídem, pero esta vez usando un documento indexado. De esta manera veremos la mejora en el rendimiento.

Hay que definir una consulta de cada tipo y lanzarlas, recogiendo sus tiempos.

Indique todo el procedimiento y comandos utilizados.

3º) Ejecución de las modificaciones.

Tenemos un conjunto de tipo de modificaciones:

U1: Modificación del valor de un atributo de un solo documento: Nos aseguraremos, a través de condiciones de filtrado, que solo un documento va a ser modificado.

U2: Modificación del valor de un atributo de un conjunto de documentos: Nos aseguraremos, a través de condiciones de filtrado, que múltiples documentos van a ser modificados. Cuanto mayor sea el número de documentos afectados, mejor.

Hay que definir una modificación de cada tipo y lanzarlas, recogiendo sus tiempos.

Indique todo el procedimiento y comandos utilizados.

4º) Volvemos al paso 1 con el objetivo de insertar más documentos a la colección y volver a tomar tiempos de las consultas Q1, Q2, U1 y U2. El número de veces que repetiremos el experimento depende de nosotros, pero es conveniente ir probando con distinto número de documentos en la colección para ver la progresión de los tiempos a medida que vamos aumentado dichos documentos. Al menos se han de llevar a cabo las ejecuciones de las consultas con tres números distintos de documentos en la colección.

Además de las explicaciones pertinentes relacionadas con el desarrollo del experimento, al final se pide, para cada operación ejecutada, una gráfica con la siguiente estructura:

Tiempo (ms)

Número de documentos

En esta gráfica se representará la evolución del tiempo de todas las operaciones (la carga de datos, las dos lecturas y las dos escrituras) en base al número de documentos. Debe ser comentada de forma razonada.

E2. (120 minutos – 2,5 puntos) Experimento con MONGODB IN-MEMORY.

A continuación, hay que repetir el experimento, pero con el motor IN-MEMORY de la versión de MONGODB que hemos instalado.

Lleve a cabo de nuevo todos los pasos del ejercicio anterior, llevando a cabo la toma de tiempos para todas las operaciones cada vez que aumente el número de documentos de la colección.

E3. (60 minutos – 2,5 puntos) Comparativa de tiempos.

Lleve a cabo una comparativa de tiempo entre los experimentos llevados a cabo con la base de datos en modo estándar y en modo IN-MEMORY. Para cada una de las operaciones, compare tiempos y, de forma razonada, exponga sus conclusiones. Se recomienda que se apoyen en gráficas para ilustrar los resultados.

Fecha y Modo de entrega

- Los alumnos entregarán un documento PDF a través de una actividad programada en el Aula Virtual empleando la plantilla suministrada.

- La entrega se ha de llevar a cabo durante el día 29 de junio de 2026.

Bibliografía

1. MongoDB (https://docs.mongodb.com/manual/core/inmemory/)

RESOLUCIÓN

Diseño del experimento

Entorno. MongoDB Enterprise 8.3.11 en Linux (2 vCPU, 7 GB de RAM), con dos instancias en la misma máquina: **27017 con el motor estándar WiredTiger** (datos y *journal* en disco) y **27018 con el motor In-Memory** (`--storageEngine inMemory --inMemorySizeGB 1`, arrancada como en la actividad In-Memory I). Se usa el mismo script contra ambas para que las condiciones sean idénticas.

Tamaños. Se crece la colección por pasos acumulativos: **25.000 → 50.000 → 100.000 → 200.000 → 400.000 documentos** (cinco tamaños; el enunciado pide al menos tres). En cada paso se insertan sólo los documentos que faltan hasta el nuevo total, con la función `populatePhones` del enunciado, y después se miden las cuatro operaciones:

Operación	Implementación	Qué mide
I1	`populatePhones(800, inicio, fin)`: un `insertOne` por documento	Tiempo total de la carga del lote
Q1	`find({ display: <valor> }).display` **no está indexado**	Búsqueda de un documento concreto recorriendo la colección (COLLSCAN)
Q2	`find({ _id: <valor> })._id` está indexado	Búsqueda del mismo documento por índice (IXSCAN)
U1	`updateOne({ _id: <valor> }, { $set: { revisado: true } })`	Modificación de un único documento
U2	`updateMany({ "components.country": 3 }, { $inc: { "components.prefix": 1 } })`	Modificación de ~1/8 de la colección (miles de documentos)

El documento buscado en Q1, Q2 y U1 es el último insertado en cada paso: el peor caso para el recorrido secuencial, porque está al final. Los tiempos se miden con `Date.now()` antes y después de cada operación en `mongosh`, y en las consultas se comprueban además con `explain("executionStats")` (documentos y claves examinados).

```
// Experimento GVDM14 — mide I1, Q1, Q2, U1, U2 para varios tamaños de colección
const bench = db.getSiblingDB('bench');
bench.phones.drop();
function populatePhones(area, start, stop) {
  for (var i = start; i < stop; i++) {
    var country = 1 + ((Math.random() * 8) << 0);
    var num = (country * 1e10) + (area * 1e7) + i;
    bench.phones.insertOne({ _id: num,
      components: { country: country, area: area, prefix: (i * 1e-4) << 0, number: i },
      display: "+" + country + " " + area + "-" + i });
  }
}
function t(f) { const a = Date.now(); const r = f(); return [Date.now() - a, r]; }
const pasos = [25000, 50000, 100000, 200000, 400000];
let actual = 0; const base = 5550000; const res = [];
for (const total of pasos) {
  const [ti] = t(() => populatePhones(800, base + actual, base + total));
  const insertados = total - actual; actual = total;
  // documento objetivo: el último insertado (se busca por display, NO indexado)
  const obj = bench.phones.find().sort({ $natural: -1 }).limit(1).next();
  const [tq1, n1] = t(() => bench.phones.find({ display: obj.display }).toArray().length);
  const eq1 = bench.phones.find({ display:
obj.display }).explain('executionStats').executionStats;
  const [tq2, n2] = t(() => bench.phones.find({ _id: obj._id }).toArray().length);
```

```
  const eq2 = bench.phones.find({ _id: obj._id }).explain('executionStats').executionStats;
  const [tu1, r1] = t(() => bench.phones.updateOne({ _id: obj._id }, { $set: { revisado:
true } }));
  const [tu2, r2] = t(() => bench.phones.updateMany({ 'components.country': 3 }, { $inc:
{ 'components.prefix': 1 } }));
  const fila = { docs: total, insertados: insertados, I1_ms: ti, Q1_ms: tq1, Q1_docsExam:
eq1.totalDocsExamined, Q1_explain_ms: eq1.executionTimeMillis,
    Q2_ms: tq2, Q2_keysExam: eq2.totalKeysExamined, Q2_explain_ms: eq2.executionTimeMillis,
    U1_ms: tu1, U1_mod: r1.modifiedCount, U2_ms: tu2, U2_mod: r2.modifiedCount };
  res.push(fila); print(JSON.stringify(fila));
}
print('MOTOR=' + db.serverStatus().storageEngine.name);
```

E1. Experimento con el motor estándar (WiredTiger)

```
aramber@localhost:~$ mongosh --quiet --port 27017 bench.js | tee res_wiredtiger.jsonl
{"docs":25000,"insertados":25000,"I1_ms":11120,"Q1_ms":10,"Q1_docsExam":25000,"Q1_explain_ms":8,"Q2_ms":1,"Q2_keysExam
":1,"Q2_explain_ms":0,"U1_ms":2,"U1_mod":1,"U2_ms":32,"U2_mod":3125}
{"docs":50000,"insertados":25000,"I1_ms":11034,"Q1_ms":18,"Q1_docsExam":50000,"Q1_explain_ms":17,"Q2_ms":1,"Q2_keysExa
m":1,"Q2_explain_ms":0,"U1_ms":1,"U1_mod":1,"U2_ms":66,"U2_mod":6213}
{"docs":100000,"insertados":50000,"I1_ms":21214,"Q1_ms":34,"Q1_docsExam":100000,"Q1_explain_ms":34,"Q2_ms":1,"Q2_keysE
xam":1,"Q2_explain_ms":0,"U1_ms":1,"U1_mod":1,"U2_ms":171,"U2_mod":12537}
{"docs":200000,"insertados":100000,"I1_ms":42215,"Q1_ms":74,"Q1_docsExam":200000,"Q1_explain_ms":67,"Q2_ms":1,"Q2_keys
Exam":1,"Q2_explain_ms":0,"U1_ms":1,"U1_mod":1,"U2_ms":256,"U2_mod":25025}
{"docs":400000,"insertados":200000,"I1_ms":83737,"Q1_ms":121,"Q1_docsExam":400000,"Q1_explain_ms":130,"Q2_ms":1,"Q2_ke
ysExam":1,"Q2_explain_ms":0,"U1_ms":2,"U1_mod":1,"U2_ms":509,"U2_mod":49918}
MOTOR=wiredTiger
```

Figura 1. Ejecución del experimento contra la instancia estándar (puerto 27017). Una línea JSON por tamaño de colección.

Documentos	Insertados	I1 (ms)	Q1 (ms)	Q1 docs exam.	Q2 (ms)	U1 (ms)	U2 (ms)	U2 docs mod.
25.000	25.000	11.120	10	25.000	1	2	32	3.125
50.000	25.000	11.034	18	50.000	1	1	66	6.213
100.000	50.000	21.214	34	100.000	1	1	171	12.537
200.000	100.000	42.215	74	200.000	1	1	256	25.025
400.000	200.000	83.737	121	400.000	1	2	509	49.918

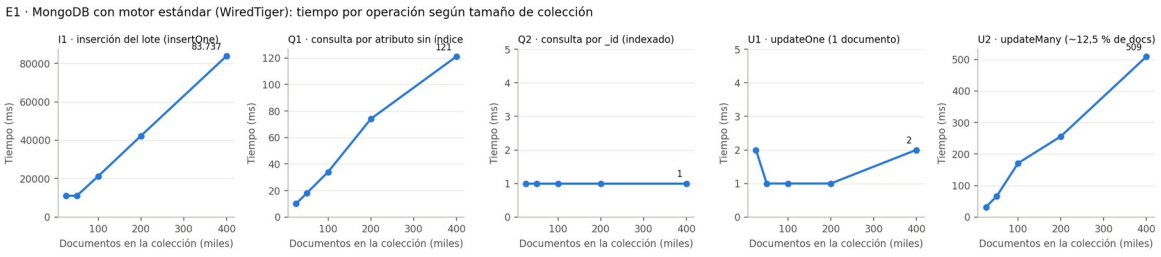


Figura 2. Evolución del tiempo de cada operación con el tamaño de la colección (motor WiredTiger).

Comentario de la gráfica:

- **I1 (carga):** crece de forma **lineal** con el número de documentos del lote: unos 11 s por cada 25.000 documentos, es decir, **≈ 0,42 ms por documento** y ≈ 2.400 inserciones/s. El coste por documento es constante: lo domina el viaje de ida y vuelta cliente-servidor de cada `insertOne`, más la escritura en caché y *journal*. El tamaño de la colección no influye.
- **Q1 (sin índice):** también **lineal con el tamaño total**: de 10 ms con 25.000 documentos a 121 ms con 400.000. `explain` confirma que examina todos los documentos (`totalDocsExamined` = tamaño de la colección).

- **Q2 (por `_id`): constante, alrededor de 1 ms**, con 1 clave y 1 documento examinados en todos los tamaños. El B-tree del índice crece en altura de forma logarítmica y eso no se aprecia en milisegundos.
- **U1:** constante (1-2 ms). Localiza el documento por índice y modifica un solo registro.
- **U2: lineal**: de 32 ms a 509 ms. Crece porque cada vez hay más documentos que cumplen el filtro (de 3.125 a 49.918) y porque el filtro por `components.country` no está indexado y obliga a recorrer toda la colección.

E2. Experimento con el motor IN-MEMORY

```
aramber@localhost:~$ mongosh --quiet --port 27018 bench.js | tee res_inmemory.jsonl
{"docs":25000,"insertados":25000,"I1_ms":11104,"Q1_ms":8,"Q1_docsExam":25000,"Q1_explain_ms":8,"Q2_ms":1,"Q2_keysExam"
:1,"Q2_explain_ms":0,"U1_ms":2,"U1_mod":1,"U2_ms":31,"U2_mod":3077}
{"docs":50000,"insertados":25000,"I1_ms":10671,"Q1_ms":18,"Q1_docsExam":50000,"Q1_explain_ms":17,"Q2_ms":1,"Q2_keysExa
m":1,"Q2_explain_ms":0,"U1_ms":1,"U1_mod":1,"U2_ms":66,"U2_mod":6244}
{"docs":100000,"insertados":50000,"I1_ms":20427,"Q1_ms":33,"Q1_docsExam":100000,"Q1_explain_ms":32,"Q2_ms":3,"Q2_keysE
xam":1,"Q2_explain_ms":0,"U1_ms":1,"U1_mod":1,"U2_ms":118,"U2_mod":12618}
{"docs":200000,"insertados":100000,"I1_ms":40986,"Q1_ms":75,"Q1_docsExam":200000,"Q1_explain_ms":65,"Q2_ms":1,"Q2_keys
Exam":1,"Q2_explain_ms":0,"U1_ms":1,"U1_mod":1,"U2_ms":305,"U2_mod":25154}
{"docs":400000,"insertados":200000,"I1_ms":83468,"Q1_ms":119,"Q1_docsExam":400000,"Q1_explain_ms":127,"Q2_ms":2,"Q2_ke
ysExam":1,"Q2_explain_ms":0,"U1_ms":1,"U1_mod":1,"U2_ms":464,"U2_mod":50229}
MOTOR=inMemory
```

Figura 3. Ejecución del mismo experimento contra la instancia In-Memory (puerto 27018).

Documentos	Insertados	I1 (ms)	Q1 (ms)	Q1 docs exam.	Q2 (ms)	U1 (ms)	U2 (ms)	U2 docs mod.
25.000	25.000	11.104	8	25.000	1	2	31	3.077
50.000	25.000	10.671	18	50.000	1	1	66	6.244
100.000	50.000	20.427	33	100.000	3	1	118	12.618
200.000	100.000	40.986	75	200.000	1	1	305	25.154
400.000	200.000	83.468	119	400.000	2	1	464	50.229

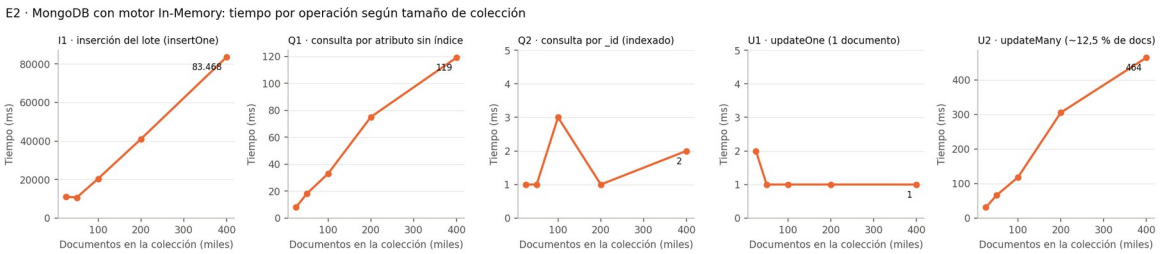


Figura 4. Evolución del tiempo de cada operación con el tamaño de la colección (motor In-Memory).

El comportamiento cualitativo es **el mismo**: I1, Q1 y U2 crecen linealmente y Q2 y U1 se mantienen en torno a 1-3 ms. Los valores absolutos son muy parecidos a los de WiredTiger, lo que se analiza en E3.

E3. Comparativa de tiempos

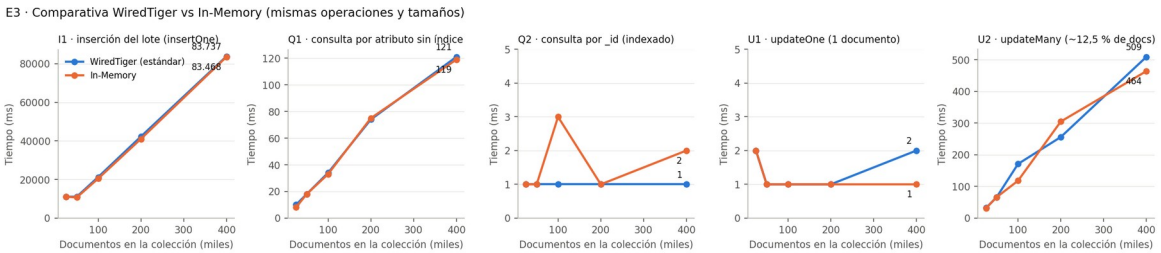


Figura 5. Comparativa WiredTiger (azul) frente a In-Memory (naranja) para cada operación.

Operación (400.000 docs)	WiredTiger	In-Memory	Diferencia
I1 – carga de 200.000 docs	83.737 ms	83.468 ms	-0.3 %
Q1 – sin índice	121 ms	119 ms	-1.7 %
Q2 – por _id	1 ms	2 ms	+100.0 %
U1 – updateOne	2 ms	1 ms	-50.0 %
U2 – updateMany	509 ms	464 ms	-8.8 %

Conclusiones por operación:

- **I1 (carga):** In-Memory es apenas un 0-4 % más rápido. El cuello de botella no es el disco sino la **latencia por petición**: 400.000 llamadas `insertOne` desde el shell. WiredTiger no escribe cada documento en disco de forma síncrona. Escribe en su caché en memoria y agrupa el *journal* en escrituras periódicas (cada 100 ms por defecto), así que con la confirmación por defecto (`w:1`, sin `j:true`) el disco apenas penaliza.
- **Q1 (sin índice):** prácticamente idéntico (121 frente a 119 ms). Con 400.000 documentos (≈ 60 MB) toda la colección cabe en la **caché de WiredTiger** (por defecto, el 50 % de la RAM menos 1 GB). Una vez cargada, WiredTiger también lee de memoria: el coste es el del recorrido y la evaluación del filtro, igual en ambos motores. El índice sigue siendo lo que marca la diferencia (Q1 frente a Q2), no el motor.
- **Q2 y U1:** ≈ 1 ms en ambos. Son operaciones puntuales resueltas por índice, por debajo de la resolución de la medida.
- **U2:** diferencias pequeñas y en ambos sentidos (In-Memory un 9 % más rápido con 400.000 documentos, más lento con 200.000), dentro de la variabilidad de la medida. De nuevo domina el recorrido en memoria.

Como el experimento del enunciado no mostraba diferencias, se añadieron **dos pruebas complementarias** para aislar el efecto del disco:

```
aramber@localhost:~$ mongosh --quiet --port 27017 bench2.js
{"motor":"wiredTiger","insertOne_w1_5000_ms":2716,"insertOne_j_true_5000_ms":3821,"insertMany_400000_ms":2124,"scan_40
0000_ms":112,"aggregate_group_400000_ms":114}
aramber@localhost:~$ mongosh --quiet --port 27018 bench2.js
{"motor":"inMemory","insertOne_w1_5000_ms":2598,"insertOne_j_true_5000_ms":"error: cannot use 'j' option when a host d
oes not have journaling enabled","insertMany_400000_ms":1841,"scan_400000_ms":114,"aggregate_group_400000_ms":113}
```

Figura 6. Prueba complementaria: escrituras con confirmación en disco (`j:true`), inserción masiva, recorrido y agregación.

```
aramber@localhost:~$ mongosh --quiet --port 27017 --eval 'db.getSiblingDB("admin").shutdownServer()' >/dev/null 2>&1; sleep 3; mongod --d
bpath /data/mongo/db --logpath /data/mongo/log/mongod.log --bind_ip 127.0.0.1 --port 27017 --fork | tail -1
child process started successfully, parent exiting
aramber@localhost:~$ mongosh --quiet --port 27017 frio.js
{"motor":"wiredTiger","docs":400000,"Q1_1a_ejecucion_ms":333,"Q1_2a_ejecucion_ms":255,"Q1_3a_ejecucion_ms":219}
aramber@localhost:~$ mongosh --quiet --port 27018 frio.js
{"motor":"inMemory","docs":400000,"Q1_1a_ejecucion_ms":203,"Q1_2a_ejecucion_ms":192,"Q1_3a_ejecucion_ms":203}
```

Figura 7. Consulta sin índice justo después de reiniciar la instancia estándar (caché fría) frente a la instancia In-Memory.

- **Escrituras duraderas:** con `writeConcern {j: true}` (esperar a que el *journal* esté en disco), 5.000 inserciones tardan 3.821 ms en WiredTiger frente a 2.716 ms sin esa espera: **+41 %**. In-Memory **rechaza** `j:true` porque no tiene *journal*. Ahí está la diferencia real: el motor en memoria es más rápido porque renuncia a la durabilidad.

- **Inserción masiva (`insertMany` en bloques de 10.000):** 2.124 ms (WiredTiger) frente a 1.841 ms (In-Memory): **In-Memory un 13 % más rápido** al eliminar la sobrecarga por petición y dejar a la vista el trabajo del motor.
- **Caché fría:** tras reiniciar WiredTiger, la primera consulta sin índice sobre 400.000 documentos tarda 333 ms y va bajando (255, 219 ms) a medida que los datos suben a caché. En In-Memory tarda ~200 ms desde la primera ejecución, porque los datos nunca salen de memoria.

Conclusión general. Con una colección que cabe en la caché y la configuración de durabilidad por defecto, **WiredTiger se comporta prácticamente como una base de datos en memoria**. Las diferencias medidas en el experimento del enunciado quedan dentro del ruido. El motor In-Memory aporta ventajas claras en tres situaciones: (1) **latencia predecible**, sin caché fría ni lecturas de disco; (2) **escrituras intensivas**, sin *journal* ni *checkpoints*, sobre todo con cargas masivas o cuando se exige durabilidad en WiredTiger; y (3) conjuntos de trabajo mayores que la caché, siempre que quepan en `inMemorySizeGB`. El precio es la **volatilidad**: en la actividad In-Memory I se comprobó que un reinicio vacía la base. Por eso en producción se usa en réplicas combinadas con miembros WiredTiger o para datos reconstruibles (cachés, sesiones, analítica en tiempo real). La mejora más grande de todo el experimento la aporta un **índice** (Q1 frente a Q2: de ~120 ms a ~1 ms), no el motor.

Referencias

[1] MongoDB Manual – In-Memory Storage Engine.
https://www.mongodb.com/docs/manual/core/inmemory/

[2] MongoDB Manual – WiredTiger Storage Engine (caché y journaling).
https://www.mongodb.com/docs/manual/core/wiredtiger/